

SOFTWARE APPLICATION PATTERNS

SYSTEM INTEGRATION

Arturo Mora-Rioja
amri@ek.dk

EK

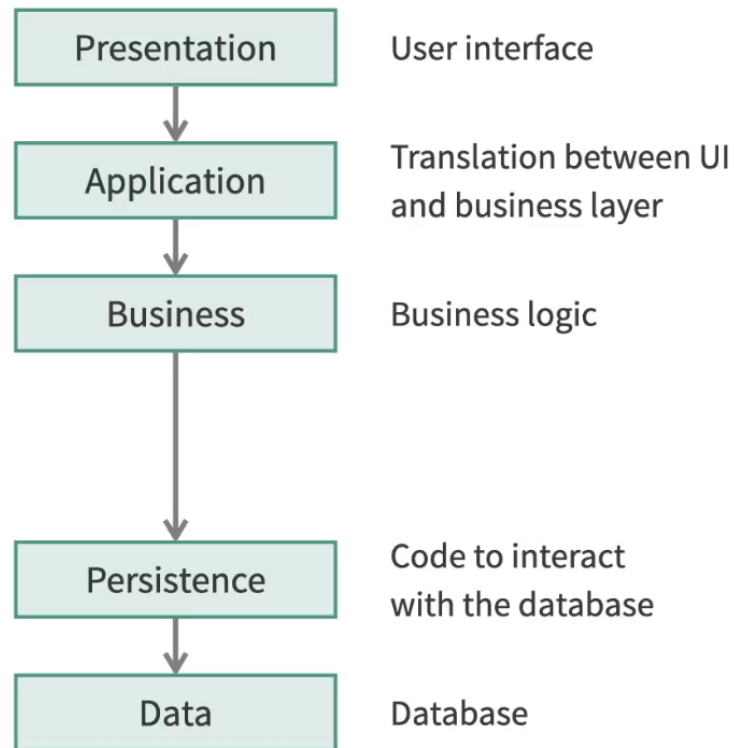
Software Application Patterns

- Layered
- Onion
- Hexagonal
- Modular monolith
- Microkernel
- CQRS
- Event sourcing

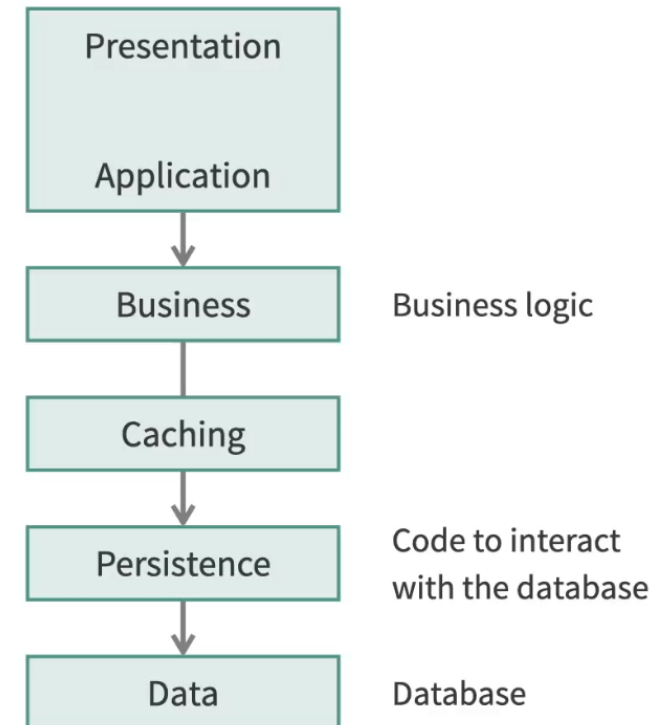


Layered

- Layers with different responsibilities
- Layers can be added and merged

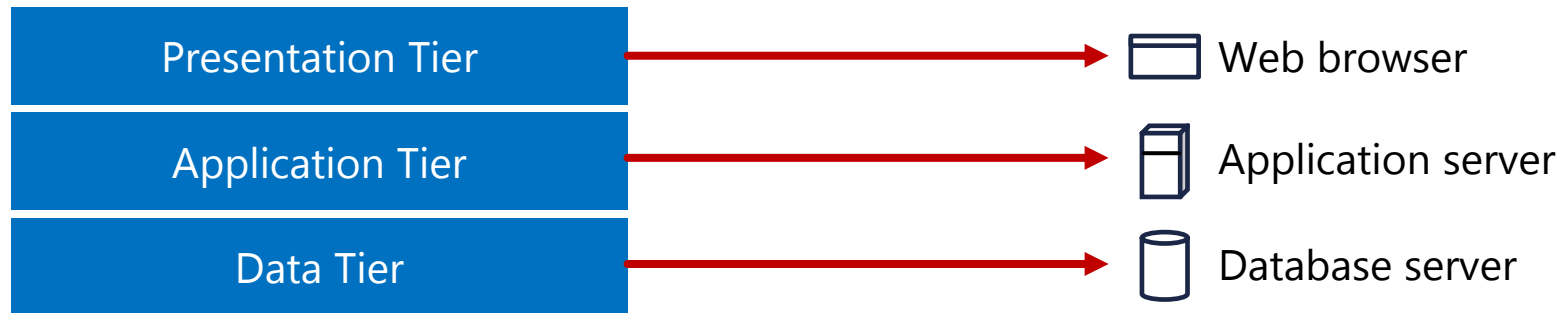


- 🙅 High coupling
 - Business logic to infrastructure
- 🙅 Too code-heavy for simple operations

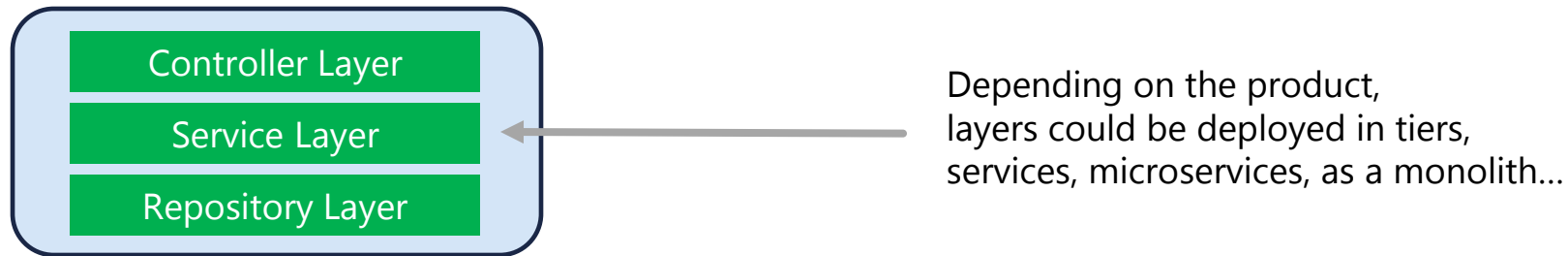


Layered Application Pattern vs. N-Tier Architecture

- N-Tier is a software architecture
 - Concerned with deployment/runtime boundaries



- Layered is a software application pattern
 - Concerned with code/logical boundaries



Layered - Use Cases

- Apps written in older versions of Java EE, COBOL, .NET
- Banking back-office systems
 - Legacy but still operational in many European banks
- Typical layering structure
 - Presentation Web GUI for employees
 - Business Account management, loan processing, anti-money laundering checks
 - Integration SOAP or REST gateways to other subsystems (fraud detection, credit scoring)
 - Data DB2, Oracle, SQL Server



© [cottonbro studio](#) @ Pexels

Layered – Use Cases (cont.)

- ERP (Enterprise Resource Planning) Systems
 - A large amount of infrastructure and domain modelling is shared
 - Stable, non-decomposable workflows (no need for microservices)
 - Centralised installation and management
- Typical layering structure
 - Presentation SAP Fiori UI, Dynamics UI, web interface
 - Application Modules for HR, finance, procurement, warehouse
 - Data Access ABAP-managed DB interfaces, repositories
 - Database SAP HANA / Oracle



Layered – Use Cases (cont.)

- Traditional Web Applications

- ASP.NET MVC, Spring MVC, Laravel, Django, Ruby on Rails
- CMSs: WordPress, Moodle
- Shopify (until they evolved into a modular monolith in 2016)
- Netflix (2000s, before they migrated to microservices)

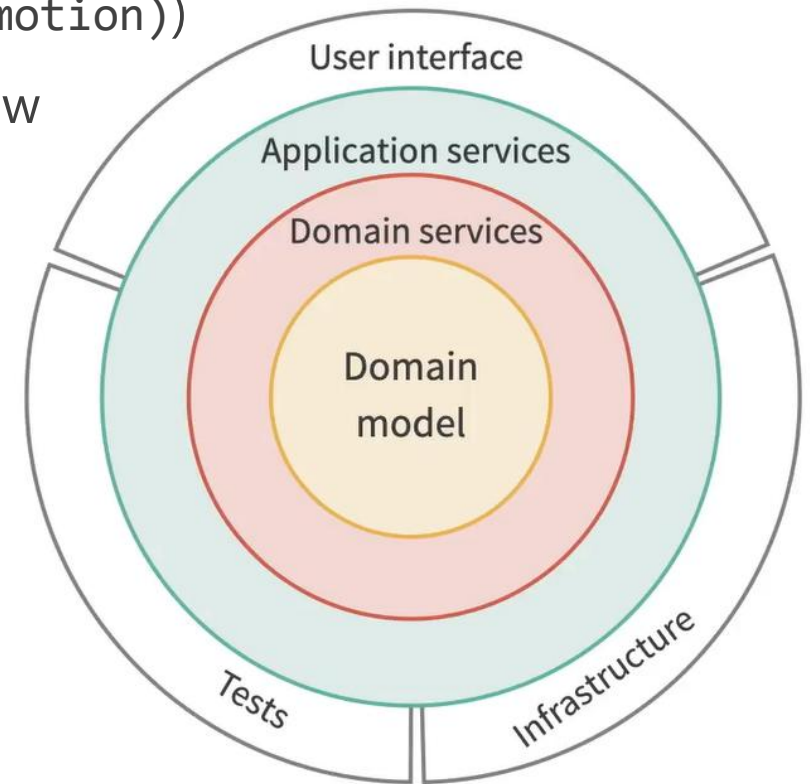


- Typical layering structure

- Presentation Controllers, templates, Razor views, Blade views, Django templates
- Business/App Service classes, domain logic, validation
- Data Access Repositories, Entity Framework / Hibernate / Eloquent models
- Data RDB (SQL Server, PostgreSQL, MySQL)

Onion

- Layers
 - **Domain model.** Business logic per entity (e.g., `Order.addItem()`)
 - **Domain services.** Business logic spanning multiple entities (e.g., `DiscountService.calculate(order, customer, promotion)`)
 - **Application services.** Use case orchestration and workflow (e.g., `PlaceOrderApplicationService`, which may involve loading customer and products, creating an order, price calculation, saving the order, publishing an event)
 - **Infrastructure.** Technical implementations (e.g., database and repository access, file systems, external APIs, logging, caching, auth providers)
- Each layer does not know how the others work
- Associated with DDD (Domain-Driven Design)



Onion - Use Cases

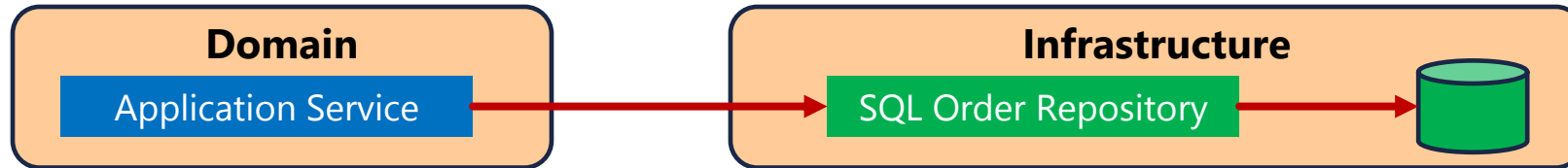
- Order processing/e-commerce systems
 - Business rules (tax, inventory, promotions), order state validation, amount calculations
- Enterprise projects with complex domains
 - Changes in regulations, compliance checks, approval workflows
- Complex healthcare systems
 - Regulatory compliance, auditability, data consistency
- SCM (Supply Chain Management) and logistics systems
 - Tracking logic, inventory rules, shipment status, route optimisation
- Subscription and membership billing platforms
 - Complex pricing models, recurring billing cycles, upgrades/downgrades, grace period

Onion

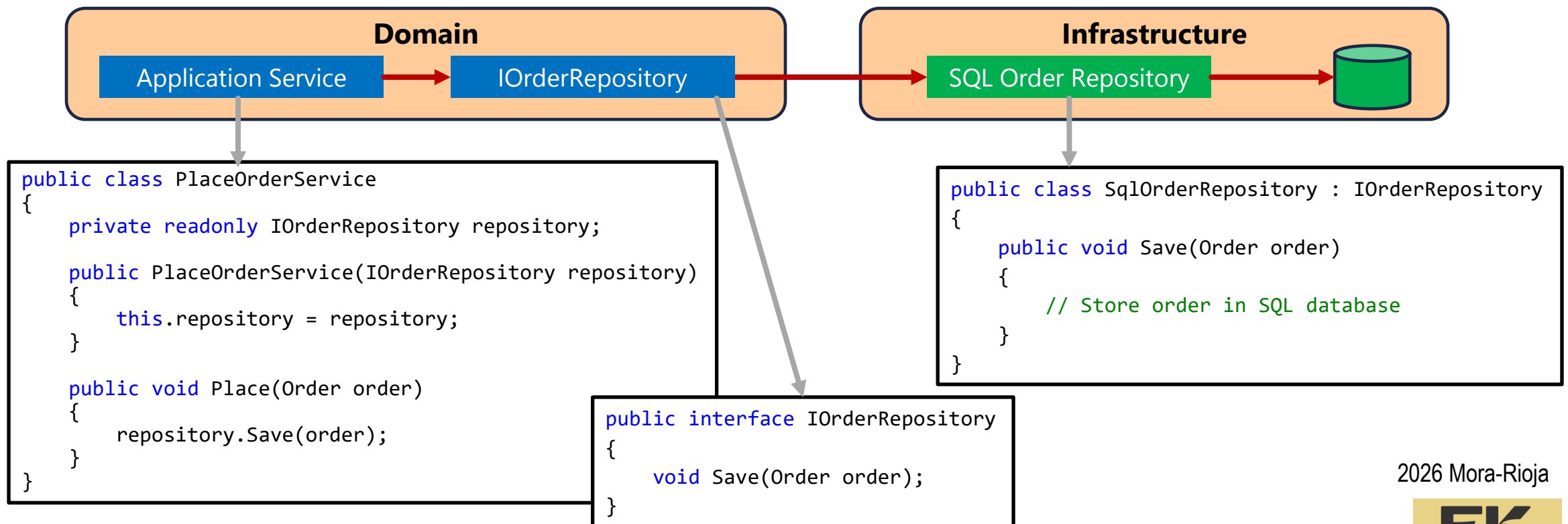
- Inner layers define interfaces; outer layers implement them (e.g., the business layer –domain model– defines the interface for a repository, but the implementation with the SQL queries will be in the infrastructure layer)
- The flow is opposite to that in the layered model
- 👍
 - The business layer is completely decoupled from the others, including infrastructure
 - Easier testing
 - Low complexity
- 👎
 - Code-heavy for large systems
 - Understanding the **dependency inversion principle** is necessary

Dependency Inversion Principle

- Without dependency inversion, the domain depends on an infrastructure implementation

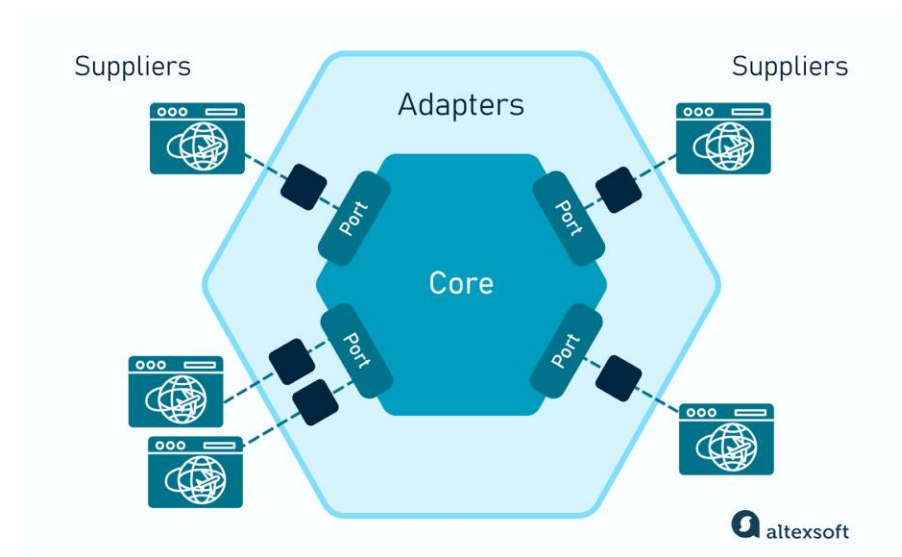


- With dependency inversion, the infrastructure layer implements a domain-defined interface



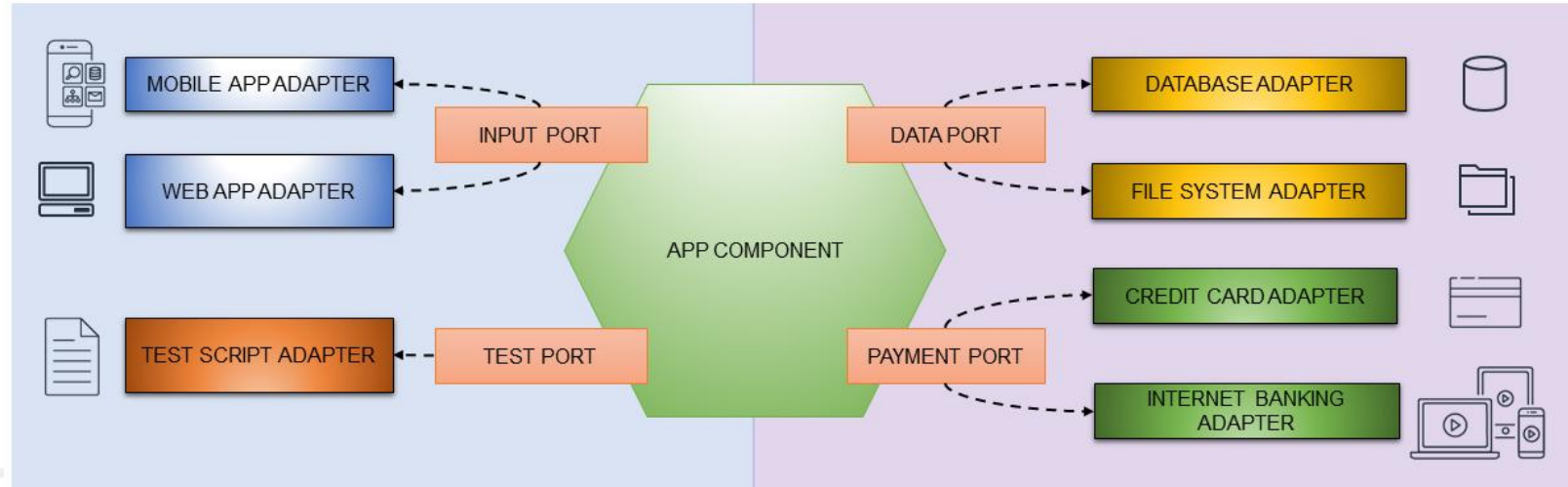
Hexagonal = Ports and Adapters

- The core module is the business logic (domain), whose interfaces are called **ports**
- The different implementations that use ports are called **adapters**
 - Primary adapters drive the application (call the ports). E.g., GUI, console app, unit tests
 - Secondary adapters are driven by the application (called by the domain)
 - E.g., BLOB storage, SQL, test doubles
- Instead of sharing an infrastructure layer, each adapter is completely independent from the rest
- 👍
 - Loose coupling
 - Reduced complexity
- 🤔
 - The domain could become a monolith
 - Understanding of the dependency inversion principle

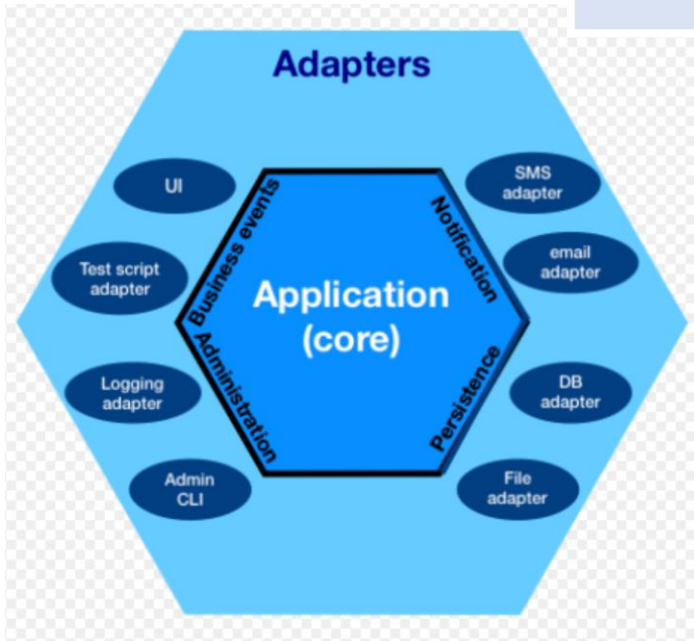


Hexagonal = Ports and Adapters

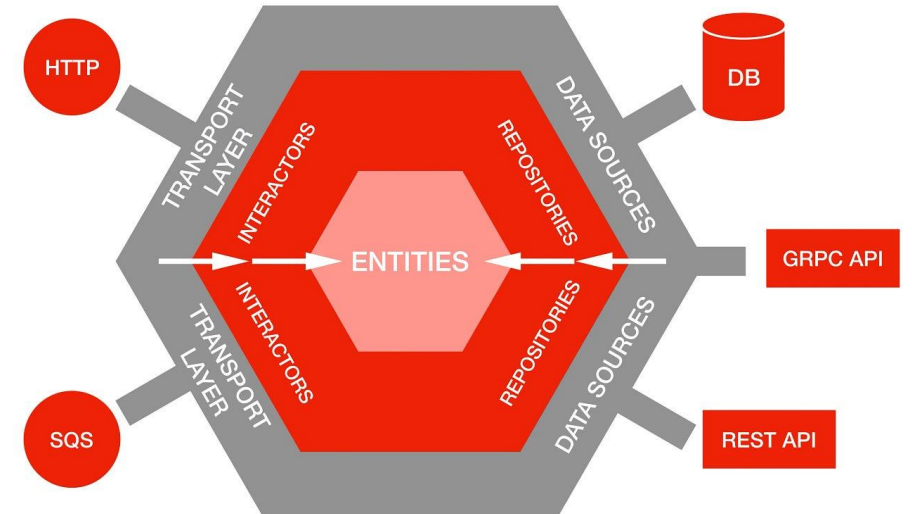
© 2025 AWS



© 2025 Gaurav Shekhar



© 2024 [Neil Avery](#)



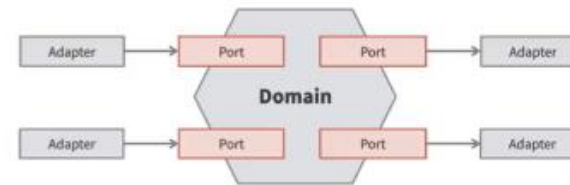
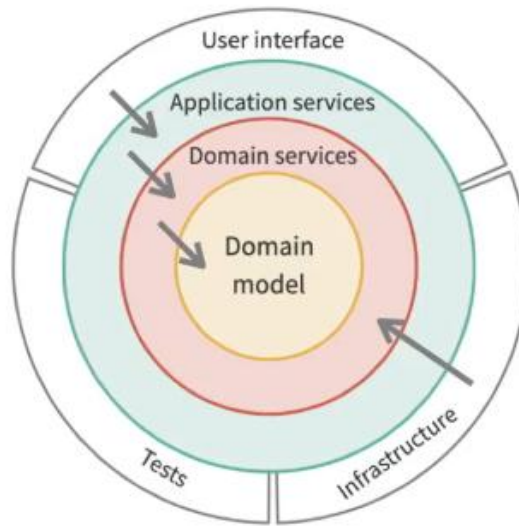
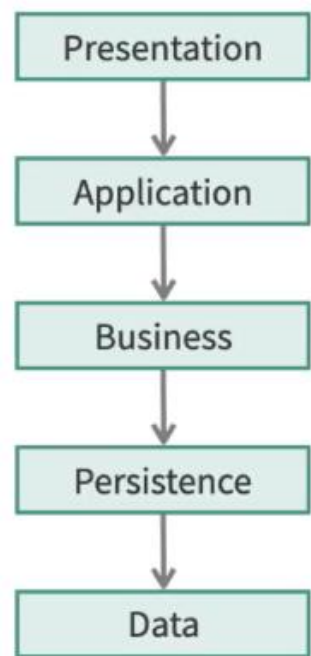
Hexagonal - Use Case

- Travel booking aggregator
 - Core service: flight/hotel search and comparison
 - Inbound port: A REST API for user searches
 - Constant integration and disconnection with different APIs
 - Outbound Ports/Adapters:
 - Some airlines' SOAP API
 - Some airlines' REST API
 - Hotel chains' XML feed
 - Different payment gateways (Stripe, PayPal)



Layered vs. Onion vs. Hexagonal

- Layered: The business domain calls the infrastructure, to which it is coupled
- Onion & Hexagonal:
An independent infrastructure calls the business domain via domain-defined interfaces
 - Onion: Constraints as to how to structure the business domain architecturally
 - Hexagonal: Free business domain implementation



Modular Monolith

- Separate modules in a single executable
- The application is divided along business boundaries (e.g., separate modules for customers, invoicing, inventory), not along technical boundaries (UI, domain, infrastructure)
- 👍
 - Loose code coupling
 - Increased agility
 - Easy to run locally
 - Easy to deploy
- 👎
 - Learning curve
 - Scaling constraints



© Anthony @ Pexels

Modular Monolith – Use Cases

- Shopify
 - Codebase organised by business domains
 - Different modules: checkout, orders, billing, ...
 - Deployed together
 - Ability to handle massive load without the complexity of distributed services
 - Modular Monolith as an alternative to microservices
- SCM systems
 - Sequential flow of well-defined domains
 - E.g., catalog → orders → inventory → logistics
- Systems built with ABP web framework for .NET



Microkernel

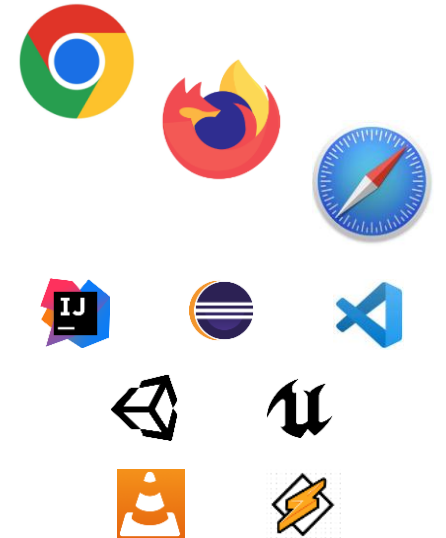
- A core module extended with plugins via a contract established by the former
- Use cases: task schedulers, workflow orchestrators, data processing apps, browser extensions
- 👍
 - Flexibility
 - Clean separation of code and teams
 - Functionality can be added and removed at runtime
- 👎
 - An extension could corrupt the core



© [Felipe López Ruiz](#) @ Pexels

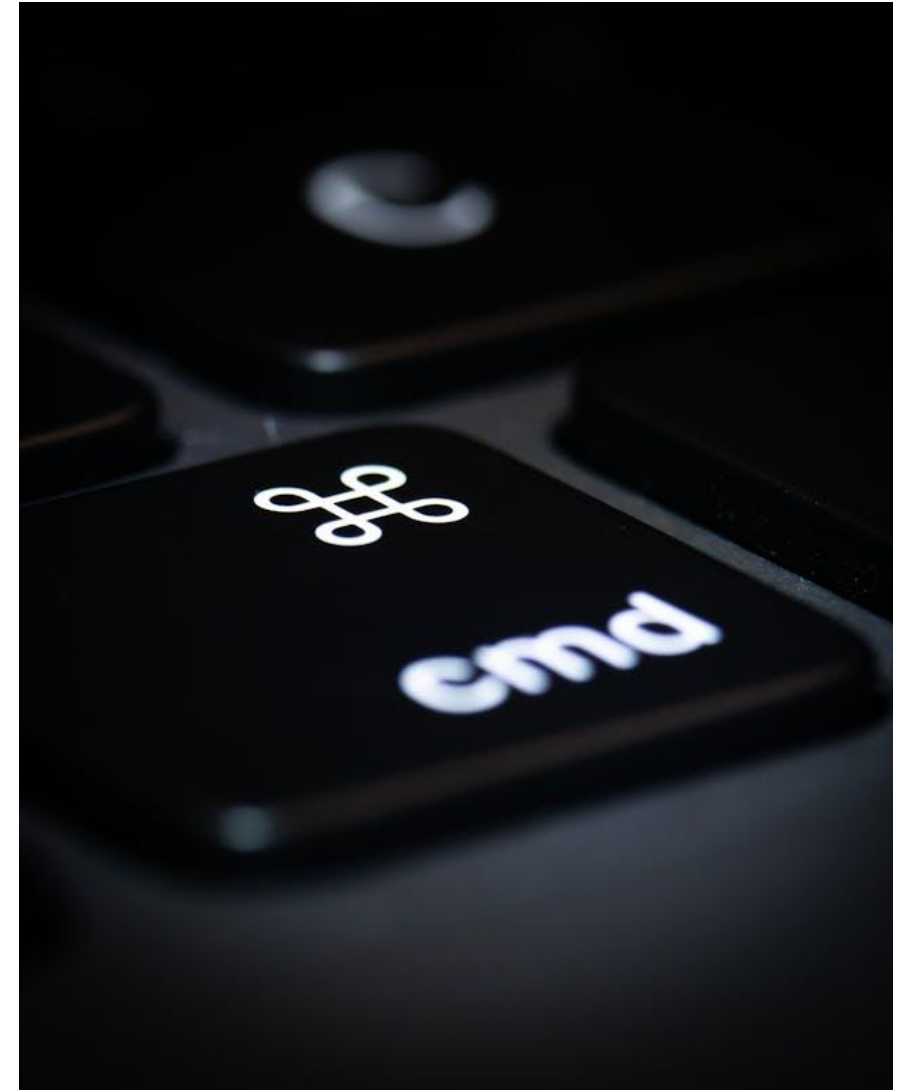
Microkernel – Use Cases

- Adobe Photoshop
 - Small, stable core module:
 - Document load/save, rendering, UI/workspace management
 - Plugins:
 - Filters, import/export formats, automation tools, scripting extensions
- Web browsers: Chrome, Firefox, Safari
 - Core: rendering, JavaScript engine, networking stack, tabs/processes, UI
 - Plugins: extensions, developer tools, protocol handlers, custom content
- IDEs: IntelliJ, Eclipse IDE, Visual Studio Code
- Game engines: Unity, Unreal Engine
- Media players: VLC, WinAmp



CQRS – Command Query Responsibility Segregation

- Two separate models for reading and writing data
 - Could involve a split database or separate databases
 - When writing
 - The write model syncs the data with the read model
 - Then the UI can get the data from the read model
- 👍
 - Reading data becomes faster and more scalable
- 🤔
 - Added complexity and learning curve
 - If using replication or sharding
 - Data duplication
 - Eventual consistency
 - Potential synchronisation issues



© [Darren Halos](#) @ Pexels

CQRS – Use Cases

- LinkedIn
 - Write responsibility: user likes, posts, shares
 - Billions of daily events stored in durable, append-only form (Apache Kafka)
 - Read responsibility: personalised feeds
 - Reading-optimised storage engines: key-value stores, graph systems, search indices
- Microsoft Azure Cosmos DB
 - Write path: indexing, consistency, replication
 - Read path: low latency, distributed queries, replicas
- Financial dashboards and real-time reporting applications
 - Write side: transactional information. Focus on data integrity
 - Read side: complex, aggregate, slow, data-intensive reports

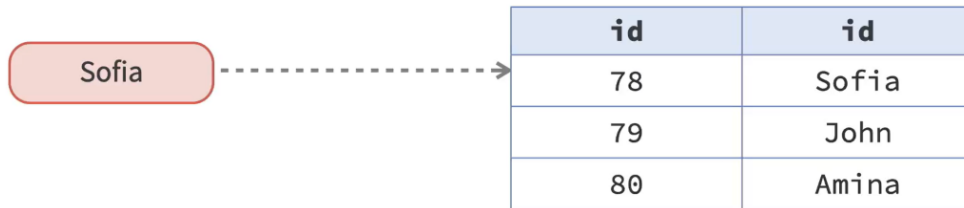


Event Sourcing

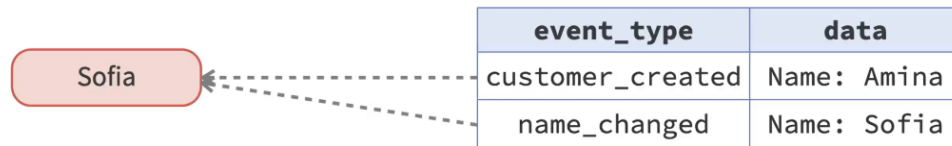
- Instead of storing data's current state, the events that happen to the said data are stored
 - Event: something that occurred in the past
- **Rehydration** = replay. To know about the current state of a data entity, all previous events must be played
- Can be used together with CQRS (event sourcing should be implemented first)
- 👍
 - Audit trail
 - Easier debugging and error fixing (events, and not data, are edited)
- 👎
 - Issues involving external systems
 - Changes in event structure invalidate previous events
 - Too many events can make replay slow. Snapshots can be used to that end

Event Sourcing Examples

- Data (present state):



- Events leading to data's present state:



© 2025 Morlion



© 2022 Comartin



Event Sourcing - Use Cases

- Uber
 - Incremental map updates
 - Dynamic pricing calculations
 - Trip lifecycle management
- Shopify
 - Orders
 - Inventory
 - Payments
- LinkedIn
 - Activity feed
 - Profile change history



Software Architectures and Application Patterns

	Layered	Onion	Hexagonal	Modular Monolith	Microkernel	CQRS	Event Sourcing
Monolith	VC	C	C	VC	C	C	C
N-Tier	VC	P	P	C	P	C	C
Ent. SOA	C	C	C	VR	P	VC	C
Microservices	C	VC	VC	VR	P	C	C
Serverless	P	P	C	VR	VR	C	C
P2P	C	P	P	P	C	P	P

Very common

Common

Possible. Context-Dependent

Very rare. Conceptual mismatch

- An application designed based on a specific architecture will implement a primary pattern, but may also implement others (usually partially)

Software Architectures and Application Patterns – Typical Pairings

Classic enterprise monolith

Architecture	Monolith	Big Java/.NET web application
Dominant pattern	Layered	UI/Controllers → Application services → Repositories → Database
Common additional patterns	CQRS	Reporting module, expensive queries
	Event Sourcing	Audit log of critical operations (e.g., bank transfers)

Well-structured monolith in a growing organisation

Architecture	Monolith	Shopify-style
Dominant pattern	Modular Monolith	Internal modules: Orders, Billing, Inventory, Users
Common additional patterns	Onion/Hexagonal	For each module (domain in the centre, infrastructure outside)
	CQRS	For modules with asymmetric read/write load

Software Architectures and Application Patterns – Typical Pairings

Traditional web application

Architecture	3-Tier	Web server, app server, and DB server hosted separately
Dominant pattern	Layered	Controllers, Services, Repositories
Common additional patterns	CQRS	Separate query services for dashboards
	Event Sourcing	For a specific subsystem (e.g., logging)

Legacy integration with enterprise bus

Architecture	Enterprise SOA	Legacy applications exposed as services
Dominant pattern	Layered	Sometimes Onion in DDD contexts
Common additional patterns	Hexagonal	Integration of newer services (adapters for SOAP, REST, MQ, ...)
	CQRS	Reporting, analytics services subscribed to events from core systems
	Event Sourcing	Long-running history-based business processes (claims, orders, logistics)

Software Architectures and Application Patterns – Typical Pairings

Domain-driven system

Architecture	Microservices	
Dominant pattern	Onion/Hexagonal	Inner: domain model. Outer: HTTP controllers, messaging, persistence
Common additional patterns	CQRS	For services with different read/write needs (e.g., Orders, Product Catalog)
	Event Sourcing	For services needing full audit/history (payments, shipping)
	Layered	For small, CRUD-based services

Cloud-managed function-based backend

Architecture	Serverless	Functions, events, managed services
Dominant pattern	Hexagonal	Incoming adapter: function handler Outgoing adapter: persistence, queues, external APIs
Common additional patterns	CQRS	Write side: command functions triggered by API gateway or events Read side: query functions accessing read-optimised stores
	Event Sourcing	Projections built by other functions

UI Patterns

- **MVC – Model-View-Controller**



- Separation of concerns
- Parallel development
- Easy to test
- Used in web frameworks



- Controllers can become bloated

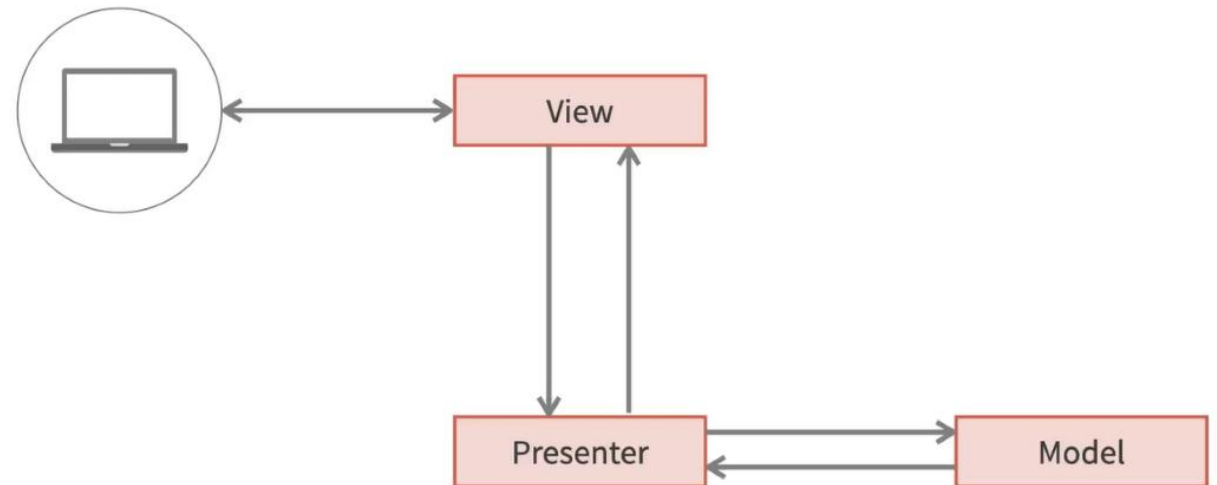


© [Godisable Jacob](#) @ Pexels

UI Patterns

- **MVP – Model-View-Presenter**

- Evolution of MVC
- 2 variations
 - Passive view. The UI logic is in the presenter
 - Supervising controller. The presenter handles the logic
- 👍
 - Separation of concerns
 - Ideal for desktop and mobile applications
 - Easy to test
- 👎
 - Presenters can become bloated



UI Patterns

- **MVVM – Model-View-Viewmodel**

- The viewmodel communicates with the view via data binding, thus with no communication at compile time
- 👍
 - Separation of concerns
 - Ideal for desktop and mobile applications
 - Easy to test
- 👎
 - Overkill for simple apps
 - Hard to debug

UI Patterns

- When to use each model
 - Server-side web applications: **MVC**
 - Ruby on Rails, Django (MTV – Model-Template-View), ASP.NET MVC, Laravel, Symfony, CodeIgniter, CakePHP, Spring MVC
 - Desktop apps with basic data binding: **MVP**
 - Android (old apps), Google Web Toolkit
 - Modern UI frameworks with reactivity and advanced data binding capabilities: **MVVM**
 - Angular (2+), Vue.js

Differences

	MVC	MVP	MVVM
User Interaction Handler	Controller	View	View
Code in UI (Code-Behind)	Minimal	Yes	Minimal
View Aware of Model?	Yes	Yes (supervising controller) No (passive view)	No
Data Binding	Basic	Basic (supervising controller) No (passive view)	Advanced

Sources

- **AltexSoft.**
["Developing Travel Booking Engines: Hands-on Experience"](#). AltexSoft, 2025.
- **AWS.**
["Hexagonal architecture pattern"](#). AWS, 2026.
- **Comartin, Derek.**
["Event Sourcing vs Event Driven Architecture"](#).
CodeOpinion, 2022.
- **Morlion, Peter.**
[*Software Architecture: Patterns for Developers*](#).
LinkedIn Learning, 2025.
- **Shekhar, Gaurav.**
["Hexagonal Architecture for Microservices Design"](#).
Medium, 2024.
- Cover picture by [mori dad](#) from [Pexels](#)